Collaborative Filtering on MovieLens 100K

Student: Raimova Alina
Team: Individual
Email: aaraimova@edu.hse.ru
Date: 2026-09-29
Assignment: Collaborative Filtering Movie Recommender (Week 3, A03)

Abstract

This report compares user-based and item-based collaborative filtering on MovieLens 100K and contrasts them with Week 2 genre recommendations. The notebook uses one per-user chronological train/validation/test split for all methods, compares three neighborhood similarity treatments, and trains a separate bias-aware matrix-factorization benchmark. On 15,851 test ratings, matrix factorization had the lowest RMSE (1.0141), while item-based mean imputation had the highest measured Recall@5 (0.0266). Lower rating error did not guarantee a stronger Top-5 list in this sparse-label experiment.

Index Terms — collaborative filtering, MovieLens, matrix factorization, recommendation evaluation

1. Introduction

Problem statement

Ratings form a mostly empty user–movie matrix, so a recommender must infer scores for movies a user has not rated. The website shows a Top-5 list from user-based CF and a Top-5 list from item-based CF.

Motivation

The experiment checks whether the two methods and their missing-value strategies produce useful predictions and lists under the same evaluation protocol.

Concrete example

User 8 rated *The Professional*, *Die Hard*, and *Star Trek: The Wrath of Khan* five stars in training. The Week 2 genre profile recommends *The Abyss* first, while default user CF recommends *Antonia's Line* and default item CF recommends *Ciao, Professore!*; Section 4 traces the scores.

Contributions

- A matched comparison of the two Week 2 methods, user CF, item CF, and matrix factorization.
- A transparent example showing how sparse rating overlap affects the scores.

2. Related Work

The MovieLens dataset paper describes how the ratings were collected and why evaluation needs care [1]. Sarwar *et al.* describe item similarity and weighted prediction as alternatives to user-neighborhood CF [2]. Koren *et al.* describe latent-factor models with user and item effects; this motivates the separate factorization benchmark [3]. The results here come from the notebook, not from those papers.

Alternatives considered

- **Co-rated only:** the website default; simple, but high similarity may rest on very few common ratings.
- **User-mean imputation:** fills the similarity matrix from training means without creating observed ratings.
- **Common-rating weighting:** shrinks similarity when overlap is small, at little extra cost.
- **Matrix factorization:** a separate rating-model benchmark rather than a neighborhood variant.

3. Method

Approach

User-based and item-based CF predict from observed ratings weighted by positive cosine similarity. A separate bias-aware factor model provides a benchmark using observed training ratings only.

Pipeline

1. Read ratings, timestamps, titles, and genres; split each user's ratings chronologically, breaking timestamp ties by item ID.
2. Fit training means, user/item similarities, and factor parameters using training ratings only.
3. Select neighborhood sizes and factor settings by validation RMSE; score the held-out test ratings and unseen-training catalog.

The same `u.data` and `u.item` files contain **100,000 ratings, 943 users, and 1,682 movies** [1]. The website shows both CF Top-5 lists for a selected user.

Key design decisions

The website's default missing-value strategy is **co-rated only**; other selector settings are exploratory. The notebook evaluates the alternatives and treats matrix factorization as a separate rating model. The website's factorization option instead fills values for similarity, so notebook results do not measure that option.

User-based CF averages observed ratings from similar users; item-based CF averages the active user's observed ratings of similar movies [2]. Both use `sum(similarity × rating) / sum(similarity)` over positive neighbors. Mean filling affects similarity only. Count weighting multiplies co-rated cosine by `min(n_common / 50, 1)`. With no usable neighbor, the fallback is the **train movie mean**, then user mean, then global mean. The factor benchmark learns global, user, and movie biases plus latent factors from observed train ratings [3].

Tools and libraries

The notebook ran with Python **3.13.7** [4], NumPy **2.3.2** [5], pandas **2.3.1** [6], and Matplotlib **3.10.5** [7]. Python runs evaluation and tests; NumPy handles similarity and factor arrays; pandas reads and summarizes ratings; Matplotlib draws data plots.

Configuration

The per-user chronological split has **69,572 train**, **14,577 validation**, and **15,851 test** ratings. Validation RMSE selected **10 user neighbors**, **160 item neighbors**, and factor rank **8**, regularization **0.02** (40 epochs, learning rate 0.15, seed 42). The common-rating threshold is configurable and was **50**.

4. Experiments

Setup

MAE and RMSE use all 15,851 test ratings for every Week 3 rating model, including fallback predictions. Top-5 Recall/NDCG use the **899 users** with a relevant test rating (≥4); all methods rank the same catalog excluding train-rated movies. Prediction coverage counts test pairs with native evidence before fallback; Top-5 catalog coverage counts movies appearing in any recommendation list. Runtime covers fitting, scoring, and metrics in one warm process, excluding loading and tuning.

Results

Table 1. User-based CF and shared factor benchmark. All ranking and catalog-coverage metrics use **Top 5**.

Method	MAE	RMSE	Recall@5	NDCG@5	Prediction coverage	Catalog coverage@5	Runtime (s)
Co-rated only	0.9039	1.1448	0.0062	0.0115	0.1689	0.1468	0.6900
User-mean imputation	0.8991	1.1439	0.0090	0.0181	0.1832	0.1082	0.6612
Common-rating weighting	0.9152	1.1732	0.0038	0.0093	0.7644	0.1706	0.6839
Matrix factorization*	**0.8066**	**1.0141**	0.0005	0.0013	0.9932	0.0268	0.9322

Table 2. Item-based CF and the same factor benchmark. Catalog coverage is measured at **Top 5**, using the same test cases.

Method	MAE	RMSE	Recall@5	NDCG@5	Prediction coverage	Catalog coverage@5	Runtime (s)
Co-rated only	0.9396	1.1795	0.0000	0.0000	0.9932	0.1296	4.9392
User-mean imputation	0.9298	1.1650	**0.0266**	**0.0539**	0.9932	0.1570	4.7664
Common-rater weighting	0.9557	1.2061	0.0000	0.0000	0.9932	0.1350	5.0879
Matrix factorization*	**0.8066**	**1.0141**	0.0005	0.0013	0.9932	0.0268	1.0292

*The same factor-model settings and training data produce equal predictions in both sections. It is a **shared benchmark**, not user-based or item-based neighborhood CF. Runtime includes each section's separate evaluation work.

Comparison with the Week 2 baseline

The two genre methods were rerun on this notebook's chronological split. All five methods rank the same unseen-training catalog for the same 899 users; numbers from Week 2's earlier leave-one-out report are not mixed in. The Week 2 genre methods produce similarity rankings, not star-rating predictions, so MAE and RMSE do not apply to them.

Table 3. Matched Top-5 comparison. Overlap is the mean number of movies shared with the Week 2 three-film profile.

Method	Recall@5	NDCG@5	Catalog coverage@5	Mean Top-5 overlap with Week 2 profile
Week 2: one movie's genres	0.0087	0.0169	0.3014	0.9900
Week 2: three-movie genre profile	0.0090	0.0183	0.2628	5.0000
Week 3: user-based CF, co-rated	0.0062	0.0115	0.1468	0.0434
Week 3: item-based CF, co-rated	0.0000	0.0000	0.1296	0.0278
Week 3: matrix factorization	0.0005	0.0013	0.0268	0.0122

Verification: User 8 worked example

User 8 has **41 train ratings** and **six relevant test movies**. The Week 2 profile uses three train-rated 5-star films: *The Professional*, *Die Hard*, and *Star Trek: The Wrath of Khan*. Their Top-5 lists are:

Method	User 8 Top-5, in rank order
Week 2: one movie's genres	*Bound*; *Amateur*; *Copycat*; *Desperate Measures* (ID 329); *Desperate Measures* (ID 348)
Week 2: three-movie genre profile	*The Abyss*; *Escape from L.A.*; *Escape from New York*; *Hard Target*; *The Lost World: Jurassic Park*
Week 3: user-based CF	*Antonia's Line*; *The Remains of the Day*; *Mystery Science Theater 3000: The Movie*; *Maya Lin: A Strong Clear Vision*; *The Spitfire Grill*
Week 3: item-based CF	*Ciao, Professore!*; *Faces*; *Of Human Bondage*; *The Saint of Fort Washington*; *Spirits of the Dead*
Week 3: matrix factorization	*Golden Earrings*; *Santa with Muscles*; *Aiqing wansui*; *Everest*; *Someone Else's America*

The single-film method ranks *Bound* first because its genres match *The Professional* exactly (cosine **1.000**). The three-film profile mixes the source genres and ranks *The Abyss* first (cosine **0.832**). User CF predicts **5.000** for *Antonia's Line* from one selected neighbor, User 52: `1.000 × 5 / 1.000 = 5.000`. Item CF predicts **5.000** for *Ciao, Professore!* from User 8's 5-star rating of *Contact*, with item similarity 1.000; that target has only **one train rating**. Factorization's first score for *Golden Earrings* is `3.6038 + 0.1993 + 1.2132 + 0.0575 = 5.0738`, clipped to **5.000**; it too has only one train rating. None of these Top-5 lists contains one of User 8's six relevant test movies, so all five have **Recall@5 = NDCG@5 = 0 for this user**. A movie without a test label is unknown, not a confirmed bad recommendation.

The saved notebook output was checked against Tables 1–3 and the User 8 trace. Its **12 automated tests** cover train-only calculations, held-out leakage, missing versus real zero ratings, weighting at 3/40/50 common ratings, candidate masking, sparse-neighbor fallback, and ranking formulas. These are agent checks; a student-performed manual check still needs to be recorded in Section 6.

5. Discussion

Failure case: The first user-based evaluation still reported Top-10 metrics although I had requested Top-5. I pointed out the mismatch; during the correction, the paired NDCG table also failed.

Root cause: The user-based default cutoff remained 10. After it was changed to 5, the paired table's column selection returned a DataFrame where the calculation expected a Series.

Fix and verification: The cutoff and paired table selection were corrected for Recall@5 and NDCG@5. The notebook then ran end to end without saved error outputs, and all 12 tests passed. I also reviewed the presentation and metrics for the different algorithms.

What surprised me / didn't work

The user similarity matrix has **943^2 = 889,249** cells; the item matrix has **$1{,}682^2$ = 2,829,124** cells, about **3.18 times larger**. Thus the usual claim that item CF scales better because there are fewer items does not hold for this dataset. The measured item-neighborhood runs were also slower than user-neighborhood runs here, although their implementations do different work, so timing alone does not isolate matrix size.

The three missing-value strategies have a simple hand check. For users `A=(5,4,missing,1)` and `B=(5,missing,2,1)`, co-rated cosine uses only movies 1 and 4 and gives **26/26 = 1.0000**. Filling each user's missing rating with their observed mean gives **0.9761**; weighting the

original cosine by `2/50` gives **0.0400**. Co-rated only is simple but can overtrust two matches; mean filling costs more dense computation and pulls profiles toward their averages; count weighting is cheap but can shrink useful weak-overlap neighbors too much. In this run, mean filling improved item-based ranking, while the selected weighting threshold reduced it.

Sparsity explains why these choices matter. The full matrix has **100,000 / (943 × 1,682) = 6.30%** observed cells; the train matrix has **4.39%**. In training, **116,194 / 444,153 = 26.16%** of user pairs share at most two movies, and the median overlap is **6**. For movie pairs, **50.90%** have no common train rater and **25.33%** have only one or two. Similarity from one shared positive rating is automatically 1, despite very little evidence. There are **108 train-cold movies** and no train-cold users; the cold movies account for **107 test ratings**, including **26 relevant ratings**. New users or movies cannot obtain personalized neighbor evidence until ratings arrive. Error is also higher for short histories and sparse movies: factor RMSE is **1.1320** for users with 1–19 train ratings versus **0.9869** for 100+, and **1.2052** for movies with 0–4 train ratings versus **0.9789** for 50+.

What worked: Matrix factorization has the best rating MAE/RMSE, while item-based mean imputation has the best measured Top-5 result. The same split and candidate catalog make these comparisons paired.

Next improvements: Tune a minimum common-rater count and a popularity-aware tie-breaker on validation, then evaluate them on a fresh held-out split. This directly targets the one-rating five-star recommendations in User 8's example.

6. AI Usage Disclosure

- **AI tools used:** OpenAI Codex assisted with code, experiments, debugging, and this report draft. Its exact client version should be checked against the authentic session export.
- **How AI was used:** Codex implemented and reran the notebook evaluation, compared saved outputs with this report, prepared the User 8 calculation, and drafted the discussion and references.
- **What I personally verified:** reviewing the experiments setup, examples and metrics.
- **What I trusted without verification:** generate code for algorithms and style fixes.
- **Session log reference:** See attached `raimova_alina_a03_session.json` for the full AI coding tool session log.

References

[1] F. M. Harper and J. A. Konstan, "The MovieLens Datasets: History and Context," *ACM Transactions on Interactive Intelligent Systems*, vol. 5, no. 4, 2015. GroupLens PDF. Accessed 2026-09-29.
[2] B. Sarwar, G. Karypis, J. Konstan, and J. Riedl, "Item-Based Collaborative Filtering Recommendation Algorithms," *Proceedings of WWW '01*, 2001. Paper. Accessed 2026-09-29.
[3] Y. Koren, R. Bell, and C. Volinsky, "Matrix Factorization Techniques for Recommender Systems," *Computer*, vol. 42, no. 8, pp. 30–37, 2009. DOI. Accessed 2026-09-29.
[4] Python Software Foundation, "Python 3.13 Documentation." Official documentation. Accessed 2026-09-29.
[5] NumPy Developers, "NumPy 2.3 Manual." Official documentation. Accessed 2026-09-29.
[6] pandas Development Team, "What's new in 2.3.1." Official documentation. Accessed 2026-09-29.
[7] Matplotlib Development Team, "Using Matplotlib 3.10.5." Official documentation. Accessed 2026-09-29.